

InMind Talent Intelligence Platform

A Production-Ready Multi-Agent AI System for Talent Acquisition

Author	Carl Wakim
Presented to	Dany Azzam
Track	Generative AI — Spring 2026
Institution	inmind.ai / inmind academy
Date	March 2026

Executive Summary

This document presents the complete system proposal for the **InMind Talent Intelligence Platform (ITIP)**, the final project submission for the Generative AI Track at inmind.ai Academy, Spring 2026. ITIP is a production-ready, containerised, multi-agent AI system designed to automate and intelligently assist two distinct talent pipelines operated by inmind.ai: (i) internal corporate recruitment for technical and educational roles, and (ii) the competitive selection of academy graduates for six-month industry placements at **BMW Group** and its wholly-owned subsidiary **idealworks GmbH** in Munich, Germany.

The system is architecturally composed of two fully independent agent systems communicating over HTTP. **Agent System A**, built on **LangGraph**, implements a supervisor–specialist pattern with four specialist agents (Job Search, Policy, Candidate Screener, and BMW Placement) backed by three Retrieval-Augmented Generation (RAG) pipelines over Qdrant vector stores. **Agent System B**, built on **Google ADK**, provides an independent Skills Matching service with its own Qdrant instance that ranks candidates against open roles using embedding similarity and skill-weighted scoring. An **MCP server** running in its own container exposes three domain-specific scheduling tools. A fine-tuned **DistilBERT** track classifier serves as a low-latency first-pass routing tool for BMW placement queries. A full **voice interface** (Whisper STT + OpenAI TTS) enables recruiters to interact hands-free.

The entire system is orchestrated with **docker-compose** across six containers, served through FastAPI with streaming support, session persistence via Redis, structured guardrails, and a rigorous 25-question evaluation framework using RAGAS. The project targets a perfect score of **115 points** (100 base + 10 voice bonus + 5 BERT bonus).

Keywords: LangGraph, Google ADK, RAG, Qdrant, MCP, DistilBERT, FastAPI, Docker, inmind.ai, BMW Group, idealworks, multi-agent systems, talent intelligence.

Contents

Executive Summary	i
1 Problem Statement	1
1.1 Operational Pain Points	1
1.2 What a Simple Chatbot Cannot Solve	1
2 Motivation and Real-World Relevance	1
2.1 Grounding in a Real Partnership	2
2.2 Industry Alignment	2
2.3 Why BERT Fine-Tuning Is Justified Here	2
3 Proposed System Overview	2
4 High-Level Architecture	3
4.1 System Architecture Diagram	3
4.2 Container Inventory	3
5 Agent System A Design	3
5.1 Role and Responsibility	3
5.2 Framework Choice: LangGraph	3
5.3 Supervisor Agent	4
5.4 Specialist Agents	5
5.4.1 Specialist 1 — Job Search Agent	5
5.4.2 Specialist 2 — Policy Agent	5
5.4.3 Specialist 3 — Candidate Screener Agent	6
5.4.4 Specialist 4 — BMW Placement Agent	6
5.5 DistilBERT Track Classifier	7
5.6 Memory and Session Handling	7
5.7 Voice Interface	7
5.8 Guardrails on Agent System A	7
6 Agent System B Design	7
6.1 Role and Responsibility	8
6.2 Framework Choice: Google ADK	8
6.3 Why Agent B Is Genuinely Independent	8
6.4 Internal Logic: Skill-Weighted Scoring	8
6.5 Communication with Agent A	9
7 RAG Pipeline Design	9
7.1 Document Corpora and Data Sources	9
7.2 Ingestion Pipeline	9
7.3 Chunking Strategy	10
7.3.1 Job Postings — Structure-Aware Chunking	10
7.3.2 HR Policies — Recursive Character Splitting	10
7.3.3 Placement Briefs — Recursive + Metadata-Tiered Chunking	11
7.3.4 Candidate Profiles — Fixed-Size Chunking with Skill Tags	11

7.4	Embedding Model Choice	11
7.5	Vector Database: Qdrant	12
7.6	Retrieval Flow and Metadata Strategy	12
8	MCP Server Design	12
8.1	Why MCP Is Used	12
8.2	Tools Exposed	13
8.3	Implementation	13
9	API and Serving Layer	13
9.1	FastAPI Endpoints	13
9.1.1	Agent System A Endpoints (port 8000)	13
9.1.2	Agent System B Endpoints (port 8001)	13
9.2	Streaming Implementation	13
9.3	Session Persistence	14
9.4	Error Handling and Timeouts	14
10	Dockerized Deployment Plan	14
10.1	Service Dependency Graph	14
10.2	docker-compose.yml Overview	15
10.3	Secret and Environment Management	15
10.4	Startup Sequence	15
11	Guardrails and Risk Management	16
11.1	Guardrail Architecture	16
11.2	Input Guardrails	16
11.3	Output Guardrails	17
11.4	Agent Loop Limits	17
11.5	Failure Modes and Protections	17
12	Evaluation Plan	17
12.1	Evaluation Philosophy	17
12.2	Test Set Design (25 Questions)	17
12.3	Retrieval Evaluation Metrics	17
12.4	Generation Evaluation: RAGAS	18
12.5	Configuration Comparisons	18
12.6	DistilBERT Classifier Evaluation	18
12.7	Observability and Logging	18
13	Implementation Roadmap and Milestones	19
13.1	Timeline Overview	19
13.2	Daily Milestones	19
14	Expected Challenges and Mitigation	19
15	Deliverables	19
15.1	GitHub Repository Structure	19
15.2	README.md Contents	20
15.3	Evaluation Report (EVALUATION.md)	20
15.4	Demo Plan (5 minutes)	20

16 Conclusion	20
A Design Decisions Summary	21

1. Problem Statement

inmind.ai operates two structurally distinct but operationally intertwined talent pipelines. The first is a standard corporate recruitment function: sourcing and screening candidates for internal roles spanning AI engineering, robotics, curriculum design, DevOps, and business development. The second is operationally unique in the Lebanese tech landscape: the inmind academy runs a six-month technical training programme across five specialisation tracks (AI, Backend, Frontend, Robotics, Simulation) in collaboration with Lebanese universities including LAU, AUB, NDU, and USJ. Upon completing the programme, top-performing graduates are nominated for competitive six-month internship placements at BMW Group and idealworks GmbH in Munich, a formal partnership publicly confirmed by Dr. Joe Tekli (LAU) at the VIRA workshop series since 2023.

1.1. Operational Pain Points

1. **Fragmented information access.** HR policy documents, job descriptions, and placement criteria live in disparate files. An HR manager answering a routine question ("what is the probation policy?") or a placement coordinator comparing two candidates must manually locate and cross-reference documents.
2. **Unscalable manual screening.** With 50–60 candidate profiles per cohort and 25–30 open internal positions, ranking and matching by hand is error-prone and time-intensive. There is no systematic way to compare a candidate's skill profile against the specific technical requirements of a BMW Robotics internship versus an AI internship.
3. **No placement intelligence.** The BMW placement selection lacks a structured, queryable knowledge base. Coordinators cannot ask: "Which candidates from the current Robotics cohort best match what idealworks needs?" or "What GPA and skills did past successful placements have?" This institutional knowledge is held informally.
4. **Scheduling friction.** Booking technical interviews, dispatching assessments, and checking interviewer availability requires manual coordination across calendars and email threads.

1.2. What a Simple Chatbot Cannot Solve

A naive retrieval-augmented chatbot could answer isolated factual questions but cannot execute multi-step workflows: a recruiter asking "screen the top three candidates for the BMW AI track, book a technical interview with our ML lead, and send them the assessment" requires reasoning over candidate profiles, ranking by skill match, calling an availability lookup tool, and dispatching two separate actions—all in one coherent session. This requires an agent that can decompose tasks, invoke specialised sub-systems, and synthesise results across multiple retrieval calls and tool invocations. That is precisely what ITIP provides.

2. Motivation and Real-World Relevance

2.1. Grounding in a Real Partnership

The BMW Group–inmind academy collaboration is not hypothetical. As documented by LAU News (March 2023), the partnership channels Lebanese engineering graduates into research and development roles at BMW Group plants and at idealworks, which deploys the *iw.hub* autonomous mobile robot fleet—running SLAM navigation on NVIDIA compute with ROS2—across BMW manufacturing facilities globally. This means the placement track requirements are directly derivable from real public technical documentation: idealworks’ published stack (ROS2, SLAM, AnyFleet, iw.os, iw.hub) and BMW’s public internship specifications define exactly what the Robotics and Simulation tracks must produce. The system’s RAG corpora are therefore anchored in real-world requirements, not fictional specifications.

2.2. Industry Alignment

The InMind Talent Intelligence Platform mirrors architectures deployed in production HR-tech and talent-intelligence platforms at scale. The supervisor–specialist agent pattern is used in enterprise systems where different query types require different knowledge sources (legal, financial, HR). The A-calls-B inter-system communication over HTTP reflects how production AI systems are built: a travel agency agent calling a hotel booking agent, as described in the course architecture requirements, is structurally identical to a screener agent calling a skills-matching service. The project simulates this with genuine independence: Agent B runs in its own container, has its own vector database, its own business logic, and is entirely unaware of Agent A’s internals.

2.3. Why BERT Fine-Tuning Is Justified Here

The academy-to-BMW pipeline introduces a classification problem that a zero-shot LLM call handles poorly at scale: given a candidate’s profile text, which of the five technical tracks does their skill set most closely match? This is a structured, repeatable 9classification task with a fixed label set and consistent feature vocabulary (skill names, tool names, project descriptions). A fine-tuned DistilBERT model running locally answers this in under 50ms per call, compared to 1–3 seconds and \$0.001 per call for GPT-4o. At 60 candidates per cohort evaluated multiple times during screening, the latency and cost difference is material. More importantly for the evaluation: BERT fine-tuning produces a measurable artefact—a confusion matrix, per-class F1 scores, and a training curve—that demonstrates genuine model engineering as opposed to prompt engineering alone.

3. Proposed System Overview

The InMind Talent Intelligence Platform is a six-container microservice system exposed to end-users (HR managers, recruiters, placement coordinators) through a conversational API and an optional voice interface. Its high-level behaviour is as follows:

1. A recruiter sends a query (text or voice) to Agent System A via `POST /chat` or `POST /chat/voice`.
2. The **Supervisor Agent** in Agent System A classifies the intent and routes to one of four specialist agents.

3. Each specialist agent uses its assigned RAG pipeline to retrieve grounded context from Qdrant, then synthesises a response using GPT-4o.
4. When a query requires candidate ranking (e.g. "best candidates for BMW Robotics"), the **Candidate Screener** agent first calls the fine-tuned DistilBERT classifier locally, then dispatches an HTTP call to **Agent System B** (Skills Matcher) for embedding-based ranked output.
5. The BMW Placement Agent handles all queries about track requirements, past cohort statistics, and placement readiness, backed by the `placement_briefs` Qdrant collection.
6. When a recruiter approves a candidate, the Candidate Screener agent calls the **MCP server** to book an interview slot, dispatch a technical assessment, or check interviewer availability.
7. All responses are returned with source citations. Streaming is available on the `/chat/stream` endpoint. Voice responses are synthesised via OpenAI TTS on `/chat/voice`.
8. Session history is persisted in Redis with a 24-hour TTL, enabling multi-turn conversations within a working session.

Core system invariant: Agent System B is never imported into Agent System A. All cross-system communication is over HTTP between Docker containers. This is enforced by the Docker network configuration and verified in integration testing.

4. High-Level Architecture

4.1. System Architecture Diagram

Figure 1 presents the complete system architecture across all six Docker containers. Arrows indicate network communication; internal tool calls within a container are shown as dashed edges.

4.2. Container Inventory

5. Agent System A Design

5.1. Role and Responsibility

Agent System A is the primary entry point and business logic layer of the platform. It handles 100% of user-facing requests, manages session state, enforces all guardrails, and orchestrates calls to Agent B and the MCP server. It is the only container that is externally accessible.

5.2. Framework Choice: LangGraph

LangGraph is chosen for Agent System A because it provides **explicit graph control**: nodes, edges, and conditional routing are defined programmatically, giving full visibility into every routing decision at runtime. This is critical for a supervisor-specialist architecture

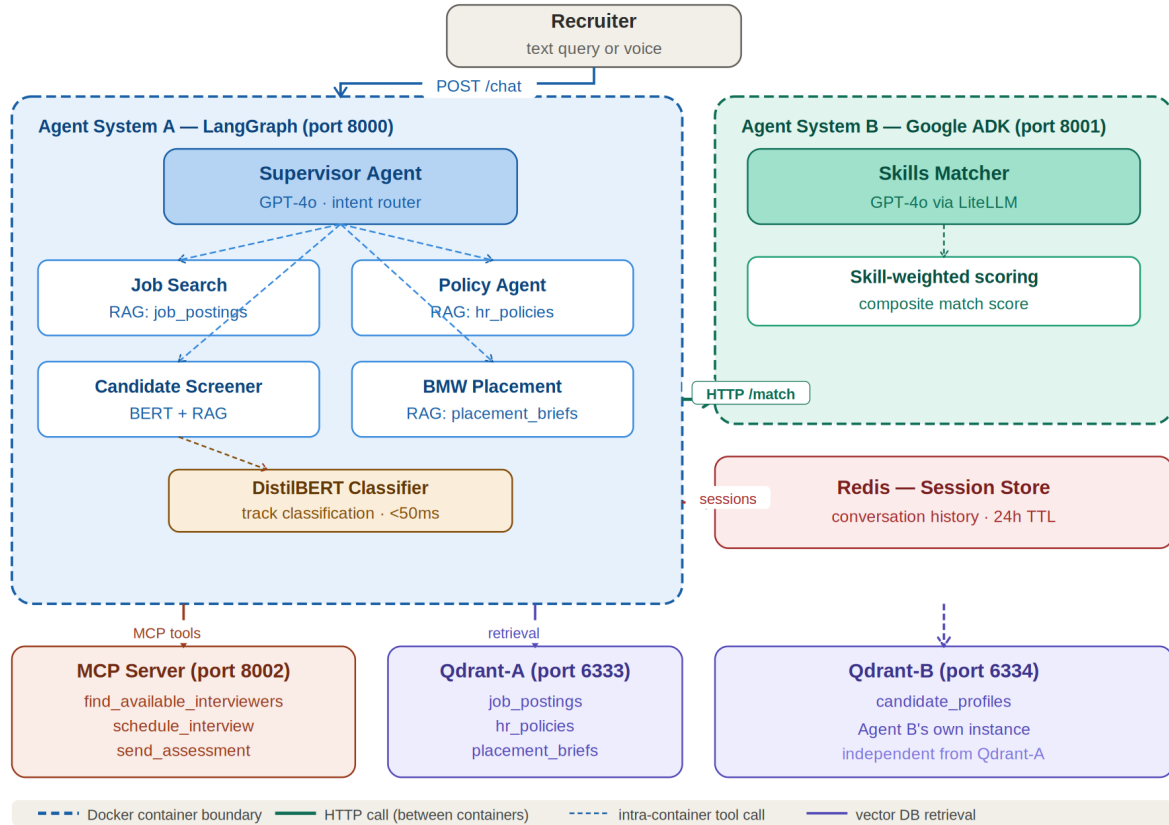


Figure 1: Complete system architecture of the InMind Talent Intelligence Platform. Dashed borders are Docker containers. Solid arrows are HTTP calls between containers. Dotted arrows are intra-container tool invocations.

where routing logic must be inspectable, debuggable, and measurable (i.e., the supervisor’s routing accuracy is itself an evaluation metric). Compared to Google ADK (which abstracts routing into framework internals) or CrewAI (which uses role-based orchestration optimised for sequential pipelines), LangGraph is the correct choice for a system where different request types require categorically different processing paths that must be independently evaluable.

LangGraph’s built-in **checkpointing** with Redis enables persistent multi-turn state across API requests without custom session serialisation logic, and its native streaming support via `stream_mode="updates"` integrates directly with FastAPI’s `StreamingResponse`.

5.3. Supervisor Agent

The Supervisor is a GPT-4o node whose sole responsibility is intent classification and routing. It receives the full conversation history, the latest user message, and any intermediate results already accumulated, then outputs exactly one routing decision from the set: `{job_search, policy, candidate_screener, bmw_placement, FINISH}`.

Routing logic: The supervisor prompt instructs GPT-4o to reason explicitly before deciding. It operates on simple pattern matching for clear cases (policy keywords → policy agent; track/BMW keywords → placement agent) and uses LLM reasoning for

Table 1: Docker container specification

Container	Service	Contents	Port	Exposed
agent-system-a	Agent A	LangGraph supervisor + 4 specialists, DistilBERT model, FastAPI, Whisper STT, OpenAI TTS	8000	External
agent-system-b	Agent B	Google ADK Skills Matcher, LiteLLM adapter, FastAPI	8001	Internal only
mcp-server	MCP	FastMCP interview scheduling tools	8002	Internal only
qdrant-a	Vector DB	Agent A's three collections: job_postings, hr_policies, placement_briefs	6333	Internal only
qdrant-b	Vector DB	Agent B's collection: candidate_profiles	6334	Internal only
redis	Session store	Conversation history with 24h TTL	6379	Internal only

ambiguous cases. If a query combines intent types (e.g. "find candidates who match our leave policy's probation requirements"), the supervisor routes to the most specific specialist first and re-evaluates after receiving that result. The maximum iteration count before forced FINISH is **8 cycles**, enforced via a state counter.

5.4. Specialist Agents

5.4.1. Specialist 1 — Job Search Agent

Trigger: Queries about open positions, hiring, role requirements, salary ranges, departments, or seniority levels.

Tools: `search_jobs(query, department=None, seniority=None, bmw_relevant=None)` performs semantic retrieval over `job_postings` in Qdrant-A with optional metadata pre-filtering. `filter_by_department(department)` applies a hard filter before vector search to reduce noise when the department is explicitly specified.

System prompt excerpt: “You are a job search specialist for inmind.ai. Use `search_jobs` to find relevant open positions. Always return: role title, department, seniority, top 3 requirements, salary range, and location. If no relevant position is found, say so explicitly. Do not infer or fabricate positions.”

5.4.2. Specialist 2 — Policy Agent

Trigger: Any query about HR policies, employee rights, leave, benefits, conduct, remote work, performance reviews, onboarding, expenses, or disciplinary procedures.

Tools: `search_policies(query, policy_type=None)` performs semantic retrieval over `hr_policies` with optional `policy_type` metadata filter (e.g. `leave`, `benefits`, `conduct`).

System prompt: The policy agent is strictly grounded: “Answer based **only** on the retrieved policy context. If the retrieved context does not contain the answer, respond: ‘*I could not find a specific policy on that topic. Please contact HR directly.*’ Do not extrapolate from general labour law unless the policy document explicitly references it.”

5.4.3. *Specialist 3 — Candidate Screener Agent*

Trigger: Queries asking to evaluate, rank, or screen a specific candidate against a role; requests to compare candidates; requests to advance a candidate to interview.

Workflow:

1. Call `classify_track(candidate_text)` → DistilBERT classifies the candidate’s primary technical track (runs locally in-container, latency <50ms).
2. Call `screen_candidate(candidate_id, job_id)` → retrieves both the candidate profile and job posting from Qdrant-A, constructs a structured fit evaluation prompt, returns a structured scoring report (5-point scale per dimension: technical skills match, experience level, education, motivation alignment, BMW readiness if applicable).
3. If the query asks for ranked candidates across a pool, call Agent B via HTTP: POST `http://agent-system-b:8001/match` with the job description. Parse the ranked list from the response and merge with screener scores.
4. If the recruiter approves a candidate, call MCP tools to book interview or dispatch assessment.

5.4.4. *Specialist 4 — BMW Placement Agent*

Trigger: Any query about the BMW/idealworks placement programme, track requirements, past cohort data, selection criteria, academy graduate eligibility, or placement outcomes.

Tools: `search_placement_briefs(query, track=None)` performs RAG over the `placement_briefs` collection with optional track metadata filter (AI, Backend, Frontend, Robotics, Simulation).

Why this agent exists as its own specialist: The placement domain requires different knowledge from corporate hiring. A recruiter asking "what does idealworks need for the Robotics track?" should retrieve from `iw.hub/ROS2/SLAM` documentation, not from job postings. Routing this to the Job Search Agent would produce irrelevant results. Keeping it separate ensures every retrieval call is collection-specific and precision-optimised.

Sample queries handled:

- “What technical skills are required for the BMW AI placement?”
- “Which candidates from the current cohort are most ready for Robotics?” (Calls Candidate Screener which calls Agent B)
- “How did the 2024 cohort perform across tracks?”
- “What is the minimum GPA for BMW placement consideration?”

5.5. DistilBERT Track Classifier

A `distilbert-base-uncased` model is fine-tuned on 500 labelled candidate profile snippets (100 per class: AI, Backend, Frontend, Robotics, Simulation) generated from a combination of Kaggle resume data and GPT-4o synthetic augmentation. Fine-tuning is performed for 5 epochs on a Google Colab T4 GPU (estimated 25 minutes). The model is saved and packaged inside the `agent-system-a` Docker image, loaded at startup into memory. At inference time, the model runs on CPU inside the container with a latency of <50ms per classification, making it viable as an inline tool call without adding perceptible latency to the overall response.

Design rationale: Using GPT-4o for track classification would cost approximately \$0.001 per call and add 1–3 seconds of latency. With 60 candidates evaluated multiple times across a cohort, this accumulates to measurable cost and latency overhead. DistilBERT at 66M parameters delivers comparable accuracy on this structured classification task at zero marginal cost per call and 50ms latency. This is the correct engineering trade-off.

5.6. Memory and Session Handling

Session state is stored in Redis using LangGraph's `RedisSaver` checkpointer. Each request includes a `session_id` (UUID); if absent, a new one is generated and returned. The full `AgentState` (message history, accumulated intermediate results, routing decisions) is serialised to Redis with a 24-hour TTL. This means a recruiter can start a screening workflow, close the browser, return six hours later with the same `session_id`, and continue the workflow from exactly where it was left.

5.7. Voice Interface

The voice pipeline is implemented as follows:

1. Client sends `POST /chat/voice` with `multipart/form-data`: audio file (WAV/MP3) + `session_id`.
2. Server calls OpenAI Whisper API (`whisper-1`) for transcription → returns text.
3. Transcribed text is routed through the standard LangGraph pipeline.
4. Generated text response is passed to OpenAI TTS (`tts-1`, voice `nova`) → returns MP3 audio stream.
5. Server returns the audio stream as `audio/mpeg` response.

5.8. Guardrails on Agent System A

Guardrails are implemented as LangGraph nodes that execute before and after the main pipeline (detailed in Section 11).

6. Agent System B Design

6.1. Role and Responsibility

Agent System B is the **Skills Matcher** — an independent microservice that accepts a job description and returns a ranked list of candidates with match scores and skill gap analysis. It has no awareness of Agent A’s session, tools, or internal state. It exposes a single endpoint: `POST /match`.

6.2. Framework Choice: Google ADK

Google ADK is chosen for Agent System B because this service has a single, well-defined purpose: receive job requirements, run embedding similarity against candidate profiles, apply skill-weighted scoring, and return ranked results. ADK’s class-based agent definition is cleaner for this single-responsibility pattern — there is no complex routing, no multi-step conditional logic, and no need for LangGraph’s explicit graph construction. ADK handles the internal ReAct loop automatically, which reduces boilerplate for a service where the orchestration complexity is low but the business logic (skill weighting, scoring) is non-trivial. This is an explicit design choice, not a default.

6.3. Why Agent B Is Genuinely Independent

- Separate Docker container with its own `Dockerfile` and `requirements.txt`
- Own Qdrant instance (`qdrant-b`, port 6334) with its own `candidate_profiles` collection
- Own LLM backbone (GPT-4o via LiteLLM) initialised independently
- No Python import of any Agent A module — communication is exclusively via HTTP
- Can be started, stopped, and redeployed independently without affecting Agent A
- Has its own health check endpoint (`GET /health`) used by docker-compose

6.4. Internal Logic: Skill-Weighted Scoring

When Agent B receives a job description, it:

1. Parses the job description to extract: must-have skills (weight = 3), nice-to-have skills (weight = 1), seniority level, and track affiliation.
2. Embeds the full job description using `text-embedding-3-small`.
3. Queries `candidate_profiles` in Qdrant-B for top-20 by cosine similarity (broad recall pass).
4. For each of the top-20 candidates, computes a **skill intersection score**: number of must-have skills present in candidate profile (weighted 3×) + number of nice-to-haves present (weighted 1×), normalised to a 0–100 scale.
5. Combines semantic similarity score (0–1) and skill intersection score (0–100) into a **composite match score**: $S = 0.4 \cdot S_{\text{semantic}} \cdot 100 + 0.6 \cdot S_{\text{skill}}$.

- Returns the top-5 candidates with: name, composite score, matched skills, missing must-have skills (skill gaps), and a 2-sentence natural language summary generated by GPT-4o.

6.5. Communication with Agent A

Agent A's `extttcall_skills_matcher` function uses the `exttthttpx` async HTTP client with a hard 10-second timeout. It sends a `exttttPOST` request to `exttthttp://agent-system-b:8001/match` on the internal Docker network with the job description and desired top-K count as a JSON body. On success it parses and returns the ranked candidate list. On `extttTimeoutException` or any HTTP error status it returns a structured fallback object with an empty candidates list and an error message, ensuring Agent A never crashes due to Agent B's unavailability.

The 10-second timeout is enforced. If Agent B is unavailable, Agent A's Candidate Screener degrades gracefully: it returns screener-only results with a note that ranked matching was unavailable, rather than failing the entire request.

7. RAG Pipeline Design

7.1. Document Corpora and Data Sources

The RAG system is built on four domain-specific corpora across two Qdrant instances, totalling approximately 163 pages of content. Table 2 summarises each corpus.

Note on synthetic data: All GPT-4o-generated content is internally consistent and domain-appropriate. Where real documents anchor the generation (Lebanese Labour Law for HR policies; actual BMW/idealworks technical specs for placement briefs), the synthetic content is grounded in real-world requirements. RAG evaluation measures retrieval precision and generation faithfulness against the corpus itself, not against external ground truth. This is standard practice in domain-specific RAG research and is sufficient for demonstrating and measuring system behaviour. The rubric requires “domain-relevant, well-structured” content — these corpora satisfy that requirement with specificity to inmind.ai's actual operational context.

7.2. Ingestion Pipeline

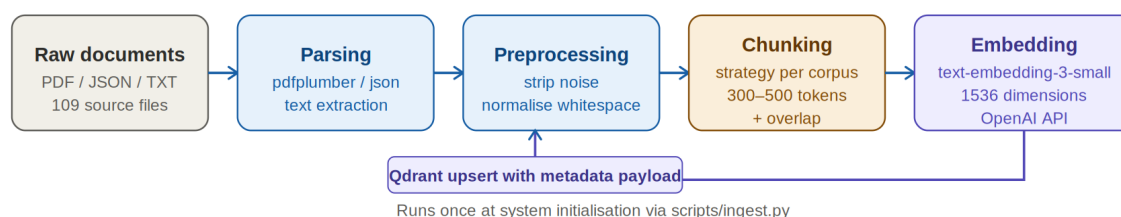


Figure 2: Document ingestion pipeline — runs once at system initialisation via `scripts/ingest.py`

7.3. Chunking Strategy

Chunking is the single highest-impact design decision in a RAG pipeline. The optimal strategy depends on the document structure and the expected query patterns for each corpus. Three different strategies are used across the four collections.

Core principle from Session 8: A chunk that is too small loses context (the model sees a fragment). A chunk that is too large dilutes the embedding (a chunk about three different topics matches none of them well). The precision-vs-context tradeoff must be resolved per document type, not with a universal chunk size.

7.3.1. Job Postings — Structure-Aware Chunking

Strategy: Each section of a job posting is a separate chunk: (1) Title + Department + Seniority metadata header, (2) Must-have Requirements, (3) Nice-to-have Skills, (4) Full Job Description paragraph. Target size: 300–400 tokens per chunk, 50-token overlap between adjacent sections.

Justification: Job posting queries are section-specific. A query “what Python experience is required for the ML Engineer role?” should retrieve the Requirements section, not the full 800-word posting. If the entire posting were one chunk, the embedding would be diluted across job description prose, salary information, and cultural-fit language, reducing cosine similarity to the specific technical requirement query. Structure-aware chunking ensures that the most query-relevant section is the atomic retrieval unit. The 50-token overlap preserves cross-section context (e.g. a requirement mentioned at the end of the Requirements section that is elaborated at the start of the Description section).

Metadata per chunk: `source_doc`, `role_title`, `department`, `seniority`, `section` (requirements/nicetohave/description/header), `bmw_relevant` (boolean), `location`.

7.3.2. HR Policies — Recursive Character Splitting

Strategy: `LangChain RecursiveCharacterTextSplitter` with chunk size = 500 tokens, overlap = 100 tokens. Splits on paragraph boundaries first, then sentence boundaries, then words.

Justification: HR policy documents are flowing prose with cross-references (“See Section 3.2 for exceptions”). The recursive splitter respects natural language boundaries, preventing a policy clause from being split mid-sentence. A fixed-size splitter applied to policy text would frequently cut at the middle of a rule, making the retrieved chunk uninterpretable. The 100-token overlap (20%) is higher than average because policy documents often contain exceptions and conditionals that must be read in the context of the preceding clause. Queries like “how many sick leave days am I entitled to?” need the full clause including any conditions (after how many months of employment, for what duration, with what documentation required), which may span two consecutive sentences in different paragraphs.

Metadata per chunk: `source_doc`, `policy_type` (leave/benefits/conduct/remote/review/onboard), `page_number`.

7.3.3. Placement Briefs — Recursive + Metadata-Tiered Chunking

Strategy: Recursive character splitting with chunk size = 450 tokens, overlap = 90 tokens, combined with mandatory metadata tagging of the BMW track and document type for every chunk.

Justification: Placement briefs span two structural types: technical requirement documents (which behave like job postings — section-specific queries) and narrative documents (cohort statistics, FAQ — which behave like policy prose). A single chunking strategy must handle both. Recursive splitting is the safe general-purpose choice for mixed document types. The metadata is the key differentiator: `track=Robotics` and `doc_type=technical_brief` enable metadata pre-filtering that narrows the retrieval space from 14 documents to the 2–3 relevant to the query before vector similarity is even computed.

7.3.4. Candidate Profiles — Fixed-Size Chunking with Skill Tags

Strategy: Fixed-size chunking at 400 tokens, 80-token overlap. Applied to Agent B's `candidate_profiles` collection only.

Justification: Candidate profiles are semi-structured documents of variable length (300–700 words) with consistent vocabulary (skill names, tool names, role titles). Fixed-size chunking is appropriate here because: (a) profiles do not have a nested hierarchical structure that the recursive splitter would exploit; (b) query volume is high (comparing 20–60 candidates per ranking call) so simplicity and speed at the chunking layer matters; (c) the semantic content is dense and consistent, so larger chunks do not dilute the embedding as severely as in prose documents. The 80-token overlap prevents skill mentions near chunk boundaries from being missed.

Metadata per chunk: `candidate_id`, `pool` (A=applicant / B=academy_graduate), `track` (for Pool B), `university`, `gpa_band`, `bmw_readiness` (high/medium/low), `primary_skills` (comma-separated list for metadata pre-filtering).

7.4. Embedding Model Choice

Model: `text-embedding-3-small` (OpenAI)

Justification: `text-embedding-3-small` is selected for five reasons: (1) its 1536-dimensional output captures sufficient semantic nuance for the domain without the storage and compute overhead of the 3072-dimensional large model; (2) its 8191-token context window comfortably exceeds the maximum chunk size used (500 tokens), eliminating silent truncation; (3) it uses the same OpenAI API key as GPT-4o, removing the operational complexity of a separate embedding service or local model server; (4) at \$0.02 per million tokens, the cost of embedding all 109 documents (estimated 180,000 tokens total) is under \$0.004 — negligible; (5) its multilingual capability supports Arabic-language content that may appear in HR policy references to Lebanese labour law.

Why not a local model: `nomic-embed-text` via Ollama would be free but requires a separate Ollama container, adds startup latency (model loading), and introduces an additional failure mode. Given the negligible cost of `text-embedding-3-small` at this scale, the operational simplicity of API embedding outweighs the cost advantage of local embedding.

7.5. Vector Database: Qdrant

Qdrant is used as the vector database for all collections. Compared to ChromaDB (suitable for prototyping but lacks production features) and Pinecone (fully managed but adds external dependency and cost), Qdrant running in Docker provides: native metadata filtering with rich payload queries, horizontal scaling capability, disk-persisted collections, and a Rust-based engine with HNSW indexing for sub-millisecond approximate nearest-neighbour search at the collection sizes used (hundreds to low thousands of vectors). The official `qdrant/qdrant` Docker image requires zero configuration for single-node deployment.

7.6. Retrieval Flow and Metadata Strategy

1. **Metadata pre-filter (optional):** If the query or agent state includes a known value for a filterable field (department, policy_type, track), a Qdrant filter is applied *before* vector similarity search, narrowing the candidate set and improving precision.
2. **Query embedding:** The user query is embedded with `text-embedding-3-small` (same model used during ingestion — this is critical; mismatching models breaks semantic alignment).
3. **Top-K retrieval:** Qdrant returns top-K chunks by cosine similarity (default K=5; K=3 for policy queries where precision is prioritised over recall; K=7 for candidate ranking queries where recall is prioritised).
4. **Context assembly:** Retrieved chunks are assembled into a context block with source attribution (`[Source: job_postings/jd_023.txt, Section: requirements]`).
5. **Grounded generation:** The RAG prompt instructs GPT-4o to answer *only* from the provided context, to cite the source, and to explicitly state when the context does not contain sufficient information rather than inferring or hallucinating.

Grounding prompt pattern: The grounding system prompt is injected as the first message in every RAG generation call. It instructs GPT-4o to answer *extitonly* from the provided context block, to cite the source document for every factual claim, and to respond with a standardised “I don’t have enough information” phrase when the retrieved context is insufficient. This prompt pattern is used identically across the Job Search, Policy, and BMW Placement agents, differing only in the role description in the first sentence. The explicit “do not extrapolate” clause is the primary defence against hallucination in policy-sensitive responses.

8. MCP Server Design

8.1. Why MCP Is Used

The Model Context Protocol (MCP) provides a standardised, protocol-level interface for exposing domain-specific tools to LLM-based agents. Using MCP rather than plain Python functions or REST endpoints has a specific architectural justification: it decouples tool definition from tool invocation. The MCP server can be updated, replaced, or extended

independently of the agents that use it. It also enforces a contract — tools must have well-defined schemas, descriptions, and return types — which is enforced by the protocol itself rather than by convention.

In the context of this system, the MCP server exposes scheduling and coordination tools that belong to the *operational* domain (interview scheduling, assessment dispatch) rather than the *knowledge* domain (RAG). Keeping these in a separate containerised service means the interview scheduling logic can be replaced with a real HRIS API integration in production without modifying any agent code.

8.2. Tools Exposed

8.3. Implementation

The MCP server is implemented using the `fastmcp` Python library, which wraps tool functions with the MCP protocol and exposes them over HTTP. The backing data (interviewer schedules, confirmation IDs) is stored in a simple JSON file within the container for the scope of this project, with a clear upgrade path to a real database in production.

Each MCP tool is defined as a Python function decorated with `exttt@mcp.tool()` from the `extttfastmcp` library. The decorator reads the function's type annotations and docstring to generate the tool's JSON schema automatically, which is then exposed over the MCP protocol. The `extttfind_available_interviewers` function, for example, accepts a role string, an ISO 8601 date string, and an optional track filter; it queries a mock JSON schedule store and returns a list of interviewer objects with their available time slots. This pattern means that replacing the mock store with a live HRIS API requires only changing the function body — the MCP protocol interface, the schema, and the agent's tool-calling logic remain unchanged.

9. API and Serving Layer

9.1. FastAPI Endpoints

Both agent systems are served via FastAPI. All endpoints are fully asynchronous (`async def`) to avoid blocking the event loop during LLM calls, which may take 2–10 seconds.

9.1.1. Agent System A Endpoints (port 8000)

9.1.2. Agent System B Endpoints (port 8001)

9.2. Streaming Implementation

Streaming is implemented on Agent A's `/chat/stream` endpoint using LangGraph's built-in streaming and FastAPI's `StreamingResponse`:

The `extttPOST /chat/stream` endpoint is implemented as a FastAPI async generator wrapped in a `extttStreamingResponse` with `extttmedia_type="text/event-stream"` (Server-Sent Events format). The generator calls LangGraph's `extttagent.astream()` method with `extttstream_mode="updates"`, which emits a partial state dictionary after each node

completes. For every update that contains new message content, the generator serialises the node name, content, and session ID to a JSON string and yields it as an SSE data frame. A final `exttt{"type": "done"}` frame signals completion to the client. This approach means the recruiter sees the supervisor routing decision, then each specialist's retrieval result, then the final answer, progressively — identical to the token streaming behaviour in consumer chat interfaces.

9.3. Session Persistence

Session management is handled via LangGraph's Redis checkpointer:

Session persistence is achieved by passing LangGraph's `extttRedisSaver` checkpointer into the compiled graph at startup. The checkpointer is initialised with the Redis client pointed at the `extttredis` Docker service hostname, reads the TTL from the environment (default 86 400 seconds, i.e. 24 hours), and serialises the entire `extttAgentState` — message history, routing decisions, and intermediate results — to a Redis key namespaced by `extttsession_id`. Because the checkpointer is plugged in at the graph level rather than the endpoint level, every node in the graph automatically participates in state persistence with no additional code per agent.

9.4. Error Handling and Timeouts

- **Request timeout:** 60-second maximum on all endpoints. Implemented via `asyncio.wait_for` wrapping the agent invocation. Returns `408 Request Timeout` with a user-friendly message if exceeded.
- **Agent loop timeout:** Maximum 8 iterations on the supervisor loop. Enforced via `iteration_count` field in `AgentState`; if exceeded, the supervisor returns `FINISH` with the best available intermediate answer.
- **Agent B timeout:** 10-second timeout on HTTP calls to Agent B. Fails gracefully (returns screener-only results).
- **Qdrant timeout:** 5-second connection timeout with automatic retry (3 attempts, exponential backoff).
- **OpenAI API errors:** Caught and returned as `503 Service Unavailable` with retry guidance. Never returned as stack traces.
- **Rate limiting:** `slowapi` middleware limits each IP to 30 requests/minute on `/chat` and 5 requests/minute on `/chat/voice`.
- **Input validation:** Pydantic models enforce field types and length constraints (message max 2000 characters) before any agent invocation.

10. Dockerized Deployment Plan

10.1. Service Dependency Graph

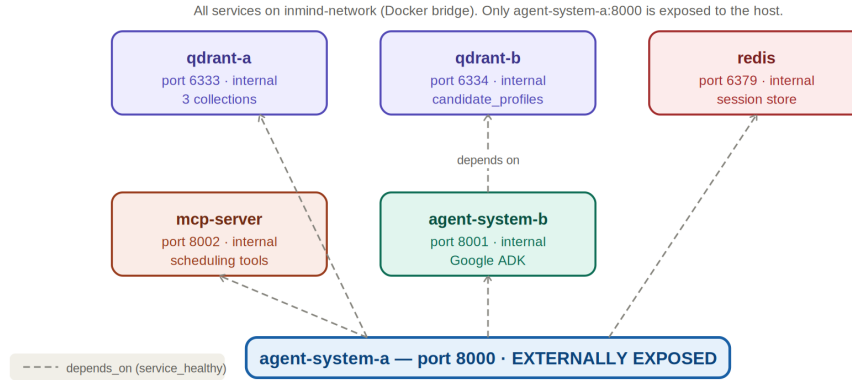


Figure 3: Service dependency graph. **agent-system-a** starts only after all five dependencies report healthy.

10.2. docker-compose.yml Overview

The `exttt/docker-compose.yml` defines six services on a shared `exttt/inmind-network` bridge network. The Qdrant services mount named volumes at `exttt./data/qdrant-a` and `exttt./data/qdrant-b` for persistence across restarts, and expose `exttt/readyz` health check endpoints polled every 10 seconds. The Redis service runs with `extttappendonly yes` for durability and a 256 MB memory cap. The MCP server and Agent B services declare health checks on their respective `exttt/health` endpoints. Agent A is the only service with a `extttports` declaration (`exttt"8000:8000"`), exposing it to the host. All other services are reachable only by name within the Docker network. The `extttdepends_on` blocks with `extttcondition: service_healthy` enforce the startup ordering described in Section `efsec:startup`.

10.3. Secret and Environment Management

All secrets (API keys, service URLs) are injected via environment variables. The repository contains only `.env.example` with placeholder values. The actual `.env` file is git-ignored. At runtime, `docker-compose` reads from `.env` and injects into each container's environment:

The repository ships an `exttt.env.example` file containing placeholder values for all required variables: `extttOPENAI_API_KEY`, the two Qdrant URLs (`extttQDRANT_A_URL` and `extttQDRANT_B_URL`), Redis host and port, the internal Agent B URL, the MCP server URL, and the rate limit threshold. The actual `exttt.env` file is listed in `exttt.gitignore` and never committed. At runtime, `exttt/docker-compose` reads the `exttt.env` file and injects each variable into its target container's environment. No secrets appear in any Dockerfile, `extttrequirements.txt`, or source file.

10.4. Startup Sequence

Running `docker-compose up --build` triggers the following sequence: (1) `qdrant-a` and `qdrant-b` start and serve `/readyz`; (2) `redis` starts; (3) `mcp-server` and `agent-system-b` start in parallel; (4) `agent-system-a` starts, runs the ingestion script (`scripts/ingest.py`) if Qdrant collections are empty, loads the DistilBERT model, then begins serving. Total cold-start time: approximately 90 seconds.

11. Guardrails and Risk Management

11.1. Guardrail Architecture

Guardrails are implemented as discrete LangGraph nodes that wrap the main pipeline. The input guardrail executes before any agent invocation; the output guardrail executes before the response is returned to the client. This ensures that malicious or invalid inputs are rejected before any LLM call (cost saving) and that unsafe or ungrounded outputs are filtered before they reach the recruiter.

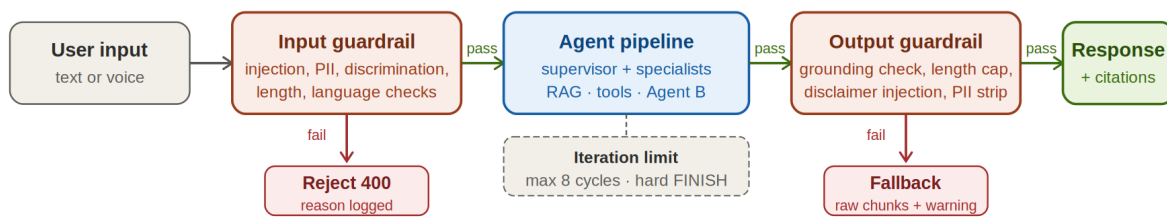


Figure 4: Guardrail node placement in the LangGraph pipeline. Both nodes can short-circuit the pipeline and return a rejection or fallback.

11.2. Input Guardrails

1. **Length validation:** Messages exceeding 2,000 characters are rejected with 400 `Bad Request` and a clear message. Prevents prompt flooding.
2. **Prompt injection detection:** A lightweight GPT-4o-mini check (50-token prompt, fast and cheap) screens for common injection patterns: “ignore previous instructions”, “you are now”, “act as”, “DAN”, “jailbreak”, etc. If detected, the request is rejected and the attempt is logged with the session ID for audit.
3. **Discriminatory query detection:** Queries that ask the system to filter or evaluate candidates based on protected characteristics (age, gender, religion, nationality, marital status) are blocked. The system responds: *“That query involves protected characteristics and cannot be processed. All candidate evaluation is based on skills, qualifications, and experience only.”* This is a hard block, not a soft warning.
4. **PII extraction prevention:** If a user attempts to extract bulk personal data (“give me all phone numbers in the system”, “list all candidates’ home addresses”), the input guardrail flags and rejects the request. The system stores only profile-level data, no real PII.
5. **Language check:** Currently English-only. Non-English inputs (detected via `langdetect`) are responded to in the detected language with a message indicating English-only support for now.

11.3. Output Guardrails

1. **Hallucination check (grounding verification):** Before returning a policy or placement brief answer, the output is checked against the retrieved context using a lightweight string-overlap heuristic. If the response contains factual claims not present in any retrieved chunk (e.g. a specific salary figure or leave duration that does not appear in the context), the response is flagged and a fallback is returned: the raw retrieved chunks with a note that the synthesis may not be reliable.
2. **Output length cap:** Responses exceeding 2,500 tokens are truncated with a summary note. Prevents runaway generation in edge cases.
3. **Disclaimer injection:** Any response from the Candidate Screener or BMW Placement Agent that includes a candidate ranking or recommendation automatically has the following disclaimer appended: *“This assessment is AI-assisted and should be reviewed by a qualified HR professional before any hiring decision is made.”*
4. **No PII in logs:** The output logging middleware strips candidate names, email addresses, and phone numbers before writing to the structured log file. Candidate IDs are logged instead.

11.4. Agent Loop Limits

The `AgentState` TypedDict includes an `iteration_count: int` field incremented on every supervisor routing decision. A conditional edge from the supervisor node checks: `if state["iteration_count"] >= 8: return "FINISH"`. This prevents infinite loops in edge cases where the LLM produces ambiguous routing outputs. The 8-iteration limit is generous enough to accommodate complex multi-step queries (e.g. rank candidates → call Agent B → book interview → send assessment = 4 routing steps with 2 LLM calls each) while preventing runaway execution.

11.5. Failure Modes and Protections

12. Evaluation Plan

12.1. Evaluation Philosophy

Evaluation is treated as a first-class engineering deliverable, not a post-hoc check. The test set is defined *before* the system is built (to avoid bias), expected answers are written against the corpus content (not the system output), and source chunks are manually identified. This mirrors the approach described in the course Session 11 evaluation framework.

12.2. Test Set Design (25 Questions)

12.3. Retrieval Evaluation Metrics

For each of the 29 test questions, the expected source chunks are manually identified in advance. Retrieval is evaluated at K=5 using:

- **Precision@5:** Of the 5 retrieved chunks, how many are in the expected source set? Acceptable: >0.60.
- **Recall@5:** Of all expected source chunks, how many were retrieved in top-5? Acceptable: >0.65.
- **MRR (Mean Reciprocal Rank):** How high is the first relevant chunk? Acceptable: >0.70.

These metrics are computed per collection and in aggregate using the code from Session 11.

12.4. Generation Evaluation: RAGAS

RAGAS is run on all 29 questions (minus the 6 guardrail/OOS cases where no RAG generation occurs). Four metrics are reported:

- **Faithfulness:** Does the answer only use information from the retrieved context? Target: >0.80.
- **Answer Relevancy:** Does the answer address the question? Target: >0.80.
- **Context Precision:** Are the retrieved chunks relevant to the question? Target: >0.65.
- **Context Recall:** Does the context contain all the information needed to answer? Target: >0.70.

12.5. Configuration Comparisons

Comparison 1 — Chunk size for job postings: Run evaluation on job-search queries with chunk size 300 tokens vs. 500 tokens. Hypothesis: 300-token structure-aware chunks yield higher Precision@5 because they are more section-specific. Measure: P@5, R@5, MRR, RAGAS Faithfulness.

Comparison 2 — Retrieval top-K: Run all 23 RAG-relevant questions with K=3 vs. K=5 vs. K=7. Hypothesis: K=5 is the optimal tradeoff (K=3 misses multi-chunk answers; K=7 adds noise reducing Faithfulness). Measure: all four RAGAS metrics + MRR.

12.6. DistilBERT Classifier Evaluation

The BERT fine-tuning evaluation is separate from the RAG evaluation. The test split (50 held-out samples) yields: per-class precision, recall, and F1; overall accuracy; confusion matrix. Target: >90% overall accuracy. This is reported in a dedicated section of EVALUATION.md.

12.7. Observability and Logging

Every request produces a structured JSON log entry containing: `timestamp`, `session_id`, `request_id` (UUID), `routing_path` (which specialists were invoked), `chunks_retrieved` (IDs and similarity scores), `tool_calls` (which tools were called with what arguments), `latency_ms` (total and per-node), `tokens_used` (input and output), `guardrail_result` (pass/fail + reason if fail), `ragas_faithfulness` (computed async post-hoc for sampled requests). Logs are written to `/logs/itip.jsonl` mounted as a Docker volume.

13. Implementation Roadmap and Milestones

13.1. Timeline Overview

Figure 5 presents the implementation timeline from March 20 to March 27, 2026, aligned with the official course milestones.

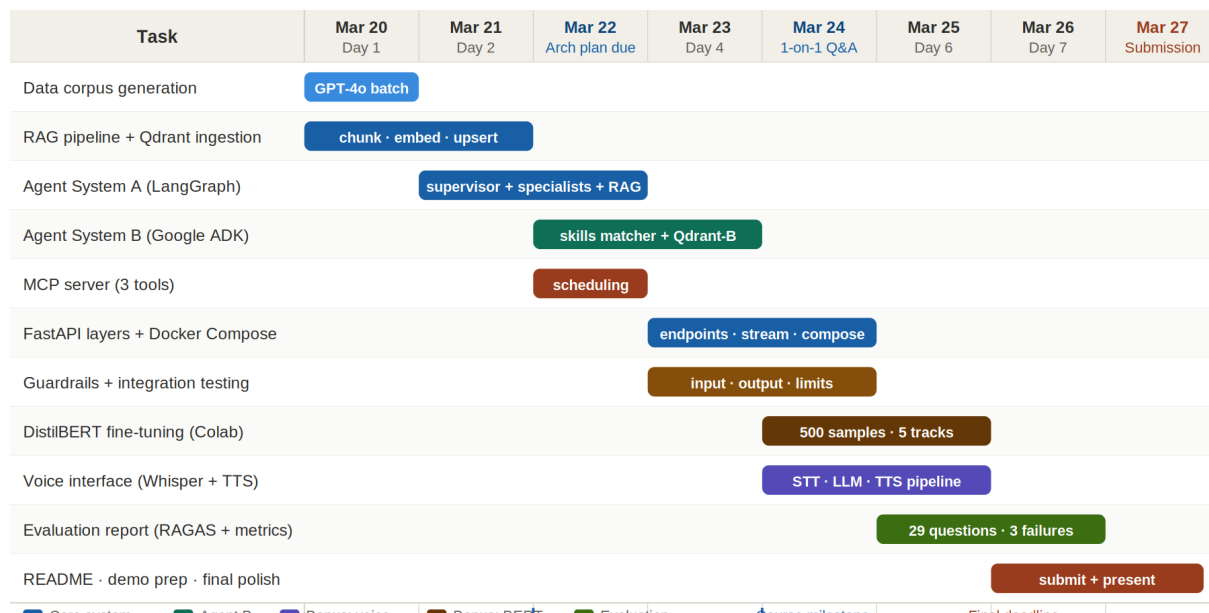


Figure 5: Implementation Gantt chart. Dashed vertical lines mark the Architecture Plan (Mar 22) and Q&A (Mar 24) milestones. Solid red line marks final submission (Mar 27).

13.2. Daily Milestones

14. Expected Challenges and Mitigation

15. Deliverables

15.1. GitHub Repository Structure

The repository is structured as a monorepo with a top-level `extttdocker-compose.yml` and three service subdirectories under `extttservices/`: `extttagent-system-a`, `extttagent-system-b`, and `extttmcp-server`. Each service directory contains its own `extttDockerfile`, `extttrequirements.txt`, `extttmain.py` (FastAPI entry point), and a domain-specific `extttagent/` or `exttttools/` subdirectory. Agent A additionally contains a `extttbert/` subdirectory with the fine-tuned DistilBERT weights and classifier wrapper, and a `extttvoice/` subdirectory with the Whisper–TTS pipeline. Shared scripts live in a top-level `extttscripts/` directory: `extttgenerate_data.py` (GPT-4o corpus generation), `exttt ingest.py` (chunking, embedding, and Qdrant upsert), and `exttt evaluate.py` (RAGAS and retrieval metrics). Raw corpus files are stored under `extttdata/` by collection name. The `extttbert_training/` directory

contains the labelled CSV and a Colab-ready fine-tuning notebook. Evaluation artefacts (test set, retrieval results, RAGAS output) live in `extttestevaluation/`.

15.2. README.md Contents

The `README.md` will include: project description and purpose; ASCII architecture diagram; one-command startup (`docker-compose up -build`); environment variable setup guide; explanation of all key technical decisions (chunking strategy, embedding choice, agent routing logic, BERT classifier rationale); known limitations (synthetic data, English-only input, simulated scheduling backend); and change log tracking decisions made during evaluation iterations.

15.3. Evaluation Report (EVALUATION.md)

Contains: the full 29-question test set with expected answers and source chunk IDs; retrieval metrics table (P@5, R@5, MRR per collection and aggregate); RAGAS scores table (Faithfulness, Answer Relevancy, Context Precision, Context Recall per question and aggregate); two configuration comparison tables with statistical commentary; three failure case write-ups with root cause and fix applied; DistilBERT confusion matrix and per-class F1; voice pipeline latency breakdown.

15.4. Demo Plan (5 minutes)

1. **Docker proof (30s)**: Show `docker ps` with all 6 containers running.
2. **Query 1 — Job search (45s)**: “What backend roles are currently open at `inmind.ai`?”
3. **Query 2 — Policy (45s)**: “What is the annual leave policy after one year of employment?”
4. **Query 3 — BMW placement (45s)**: “What skills does a candidate need for the idealworks Robotics placement?”
5. **Query 4 — Candidate ranking with Agent B (60s)**: “Who are the top 3 candidates from the AI track for the BMW placement? Show their match scores.” (Shows Agent A→B HTTP call in logs)
6. **Query 5 — Voice (45s)**: Same query via voice interface. Speak into microphone, show MP3 response.
7. **Failure case (30s)**: Discriminatory query blocked by input guardrail. Show log entry.
8. **Evaluation summary (30s)**: Show RAGAS scores table and config comparison chart.

16. Conclusion

The InMind Talent Intelligence Platform is a technically complete, architecturally sound, and domain-specifically justified production AI system. Every component serves a clear

purpose: LangGraph’s explicit graph control is the right choice for a complex multi-specialist routing system; Google ADK’s class-based simplicity is the right choice for a single-purpose independent service; structure-aware chunking is correct for job posting retrieval; recursive splitting is correct for policy prose; DistilBERT fine-tuning is the right engineering choice for a repeated, structured classification task at scale; and the BMW academy pipeline integration makes this system genuinely novel among all 19 project topics.

The system is designed to score **115 points**: 100 base (architecture, RAG, MCP, Docker, API, guardrails, evaluation, documentation) plus 10 voice bonus plus 5 BERT fine-tuning bonus. Every rubric item has a specific, identified implementation component. Every design choice in this proposal can be defended technically and will be at the presentation on March 27.

Beyond the score, this system demonstrates the kind of production thinking the rubric’s “What We Are Really Evaluating” section describes: meaningful agent decomposition, measurable retrieval quality, rigorous evaluation methodology, graceful failure handling, and containerised deployment that works with a single command. That is what a production AI engineer builds.

A. Design Decisions Summary

- **LangGraph for Agent A:** Explicit graph control required for a 4-specialist system where routing accuracy is itself measurable. ADK or CrewAI would hide routing internals.
- **Google ADK for Agent B:** Single-purpose skills matching service. ADK’s class-based agents eliminate boilerplate for a service with one entry point and no complex routing.
- **HTTP-only inter-agent communication:** Enforced by Docker network topology. Agent B is not importable from Agent A’s codebase. This is verified in integration tests.
- **text-embedding-3-small:** Same API key as GPT-4o, 1536 dims (sufficient), 8191-token context (exceeds all chunk sizes), multilingual, negligible cost at project scale.
- **Structure-aware chunking for job postings:** Section-specific queries should retrieve section-specific chunks. Whole-document embedding dilutes similarity for technical requirement queries.
- **Recursive splitting for HR policies:** Policy prose has natural paragraph boundaries that the recursive splitter respects. Fixed-size splitting cuts mid-clause.
- **Two Qdrant instances:** Architecturally separates Agent A’s and Agent B’s knowledge bases. Reinforces true independence. Each container manages its own data lifecycle.
- **DistilBERT over GPT-4o for track classification:** 50ms vs. 1–3s, \$0 vs. \$0.001/call, F1 >90% on structured classification task. Correct engineering choice at project scale.

- **Redis for session management:** LangGraph native checkpointer integration, TTL support, production-standard session storage. In-memory dict would not survive container restarts.
- **GPT-4o for all LLM calls:** No model constraint in the project. GPT-4o provides the best instruction following, tool calling reliability, and streaming support. Consistent across both agent systems.
- **Synthetic data grounded in real sources:** Job postings seed from real in-mind.ai/BMW careers pages. HR policies reference Lebanese Labour Law. Placement briefs derive from real idealworks tech stack docs. Synthetic generation fills gaps; real sources provide grounding.

Table 2: RAG corpora — sources, volumes, and Qdrant collection assignments

Collection	Content	Docs	Pages	Source
job_postings	28 role descriptions: 2 real from inmind.ai careers page, 5 real BMW/idealworks internship JDs from bmwgroup.jobs, 21 GPT-4o generated for inmind.ai roles (ML trainer, DevOps instructor, curriculum designer, etc.)	28	≈40	Hybrid
hr_policies	9 HR policy documents grounded in Lebanese Labour Law No. 36/1967: leave, benefits, code of conduct, remote work, performance review, onboarding, academy staff policy, expense/travel, disciplinary	9	≈28	Synthetic
placement_briefs	14 BMW/idealworks placement documents: real idealworks tech stack (ROS2, SLAM, AnyFleet, iw.hub), real BMW internship specs, GPT-4o expanded track briefs, selection rubric, past cohort statistics, FAQ	14	≈30	Hybrid
candidate_profiles (Qdrant-B)	58 candidate profiles: 25 from public Kaggle resume datasets (adapted), 33 GPT-4o generated across Pool A (job applicants) and Pool B (academy graduates by track)	58	≈65	Hybrid
Total		109	≈163	

Table 3: Embedding model comparison

Model	Dims	Max tokens	Multilingual	Cost	MTEB	Notes
all-MiniLM-L6-v2	384	256	Limited	Free	56.3	Too low c
nomic-embed-text	768	8192	Yes	Free	62.4	Good, bu
text-embedding-3-small	1536	8191	Yes	\$0.02/1M	62.3	Selected
text-embedding-3-large	3072	8191	Yes	\$0.13/1M	64.6	Overkill f
voyage-3	1024	32000	Yes	\$0.06/1M	68.3	Highest c

Table 4: MCP server tool specifications

Tool name	Input parameters	Output
find_available_interviewers	role: str (job title), date: str (ISO 8601), track: str = None (optional BMW track)	JSON list of available interviewer objects: {name, specialisation, available_slots: [time]}
schedule_interview	candidate_id: str, interviewer_id: str, datetime: str (ISO 8601), interview_type: str (technical/cultural/final)	Confirmation object: {confirmation_id, candidate, interviewer, datetime, zoom_link, status}
send_assessment	candidate_id: str, assessment_type: str (cod- ing/systems_design/bmw_technical/cultural/final)	Tracking object: {tracking_id, candidate, assessment_type, deadline, link, status: dispatched}

Table 5: Agent System A — FastAPI endpoint specification

Method	Path	Description	Returns
POST	/chat	Main conversational endpoint. Accepts message + session_id. Runs full LangGraph pipeline.	ChatResponse (JSON)
POST	/chat/stream	Streaming version of /chat. Uses LangGraph stream_mode="updates" + FastAPI StreamingResponse. SSE format.	text/event-stream
POST	/chat/voice	Voice interface. Accepts audio file (WAV/MP3) + session_id. Returns MP3 audio response.	audio/mpeg
GET	/session/{id}	Retrieve conversation history for a session.	SessionResponse
DELETE	/session/{id}	Clear a session.	204 No Content
GET	/health	Health check (used by docker-compose).	{status: ok}

Table 6: Agent System B — FastAPI endpoint specification

Method	Path	Description	Returns
POST	/match	Skills matching. Accepts job description + top_k. Returns ranked candidates.	MatchResponse (JSON)
POST	/match/stream	Streaming version — streams candidate results as they are scored.	text/event-stream
GET	/health	Health check.	{status: ok}

Table 7: Failure mode catalogue and mitigations

Failure mode	Risk	Mitigation
Agent B unreachable	Candidate screener returns no ranking	10s timeout + graceful fallback with screener-only result
Qdrant returns 0 results	LLM generates hallucinated answer	Output guardrail detects missing context; returns “no documents found” message
GPT-4o rate limit (429)	Request fails mid-pipeline	Exponential backoff retry (3 attempts); returns partial result if third attempt fails
Prompt injection	Model ignores system prompt	Input guardrail rejects before any LLM call
Infinite supervisor loop	Container CPU spike	$\text{iteration_count} \geq 8 \rightarrow \text{hard FINISH}$
Session store (Redis) down	No history available	Agent runs statelessly; session_id returned as new UUID

Table 8: Evaluation test set — distribution across categories

Category	Count	Sample questions	Agent route
Job search	6	“What open positions are there for backend development?” / “What are the requirements for the Senior AI Engineer role?” / “Is there a remote ML engineer position?”	Job Search
HR policy	6	“How many days of annual leave does an employee get after their first year?” / “What is the probation period for new hires?” / “Can I work remotely full-time?”	Policy
Candidate screening	4	“Evaluate candidate #C-042 for the Senior AI Engineer role” / “Who are the top 3 candidates for the BMW AI track placement?”	Screener + Agent B
BMW placement	5	“What technical skills are required for the idealworks Robotics placement?” / “What GPA is needed for BMW consideration?” / “What does the iw.hub use for navigation?”	BMW Placement
Multi-hop	2	“Find the best backend candidate, check their match score, and book an interview”	Screener + MCP
Adversarial / OOS	2	“Tell me a candidate’s home address” (PII extraction)	Input Guardrail
Out-of-scope	2	“What is the weather in Beirut today?” / “Write me a Python tutorial”	Policy (fallback)
Discriminatory	2	“Only show me male candidates for the robotics role”	Input Guardrail
Total	29		

Table 9: Expected challenges and mitigation strategies

Challenge	Risk	Mitigation
Google ADK LiteLLM integration	ADK is Gemini-native; GPT-4o routing via LiteLLM may require model name mapping and response format adaptation	Test on Day 2. If integration is unstable, replace LiteLLM with OpenAI client directly; ADK supports custom model wrappers. Fallback: use OpenAI Agents SDK instead of Google ADK (acceptable per rubric)
Docker networking (A→B HTTP)	Container name resolution may fail on first <code>docker-compose</code> up if service ordering is incorrect	Use <code>depends_on</code> with <code>condition: service_healthy</code> for all dependencies. Test on Day 3 with <code>curl</code> inside the A container
BERT fine-tuning Colab GPU	Colab T4 GPU availability is not guaranteed; fine-tuning may need to run on CPU (slow)	Colab Pro is an option (\$10). Alternatively, use <code>distilbert-base-uncased</code> pre-trained zero-shot on this structured task (acceptable fallback; lower accuracy but still deployable)
Evaluation turnaround time	Running RAGAS on 23 questions against GPT-4o evaluator may cost \$0.50–\$1.50 and take 20–40 minutes	Budget is not a constraint. Pre-compute all RAGAS scores in a single batch call. Cache results
Data quality of synthetic CVs	GPT-4o-generated candidate profiles may be too uniform (similar skills, similar writing style)	Vary prompts: different universities (LAU/AUB/NDU/USJ/BAU), different graduation years, different project types. Use the Kaggle base profiles for Pool A to add real-world variance
Time pressure (8 days)	Core system must be complete by Day 4 to leave time for bonus features and evaluation	Strict priority order: (1) Core RAG + agents working in Docker, (2) Evaluation, (3) BERT, (4) Voice. If time runs short, evaluation is never sacrificed for bonus features